ИССЛЕДОВАТЕЛЬСКИЙ ПРОЕКТ

Parallel Sort Lab

Параллельные алгоритмы сортировки

Дисциплина	Алгоритмы и структуры данных
Команда	5 участников — ФИО добавить перед сдачей
Технологии	Python 3 · Tkinter · Matplotlib · ProcessPoolExecutor

1. Цель и задачи

Цель проекта — исследовать параллельную сортировку слиянием, реализовать сравнимые варианты сортировки и измерить, при каких условиях параллельная обработка имеет практический смысл.

- Реализовать последовательный Merge Sort и Parallel Merge Sort на отдельных процессах.
- Сравнить обе реализации со встроенной функцией sorted() на одинаковых входных данных.
- Измерить время, рассчитать ускорение и эффективность, проверить корректность результатов.
- Разработать настольный интерфейс с таблицей результатов, графиками и экспортом CSV.

2. Теоретическая основа

Merge Sort использует принцип «разделяй и властвуй»: массив делится на две части, каждая часть рекурсивно сортируется, затем две упорядоченные последовательности объединяются линейным слиянием. При равенстве элементов сначала выбирается элемент левой части, поэтому алгоритм устойчив.

Рекуррентность $T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$. Дополнительная память — $O(n)$.

3. Реализации и архитектура

В проекте сравниваются три варианта. Все они получают одинаковый список целых чисел; результат каждого измерения сверяется с sorted(входные_данные). Исходный список не должен изменяться.

Вариант	Реализация	Зачем нужен
Merge Sort	Собственная рекурсивная реализация на Python	Базовая последовательная версия; O(n log n), O(n) памяти.
Parallel Merge Sort	Список делится на p частей; части сортируются через ProcessPoolExecutor, затем сливаются в главном процессе	Исследование распараллеливания и накладных расходов.
sorted()	Встроенная сортировка Python (Timsort)	Практический ориентир; это не собственная реализация проекта.

4. Параллельная обработка

Параллельный вариант создаёт пул один раз на вызов, передаёт рабочие части в процессы и выполняет заключительное попарное слияние в главном процессе. Для малых массивов ниже заданного порога используется последовательная сортировка. Для CPU-bound кода применяются процессы: потоки CPython ограничены GIL.

- Полное измеренное время включает старт пула, сериализацию и передачу данных, сортировку частей и финальное слияние.
- На macOS применяется режим запуска spawn; поэтому код запуска защищён условием if __name__ == "__main__".
- Ускорение не гарантировано: фиксированные расходы могут превышать выигрыш от параллельной работы.

5. Анализ сложности

Алгоритм	Время	Дополнительная память
Merge Sort	O(n log n) во всех случаях	O(n)
Parallel Merge Sort	Сортировка частей примерно O((n/p) log(n/p)); финальное слияние O(n log p) + расходы процессов	O(n) на главном процессе плюс копии частей
sorted() / Timsort	O(n log n); на уже упорядоченных данных возможно O(n)	O(n)

6. Основные модули

algorithms/ — сортировки; benchmark/ — генерация входа, замеры, метрики и CSV; ui/ — интерфейс и графики; main.py — запуск GUI; cli.py — консольный эксперимент; tests/ — автоматические проверки.

7. Методика эксперимента

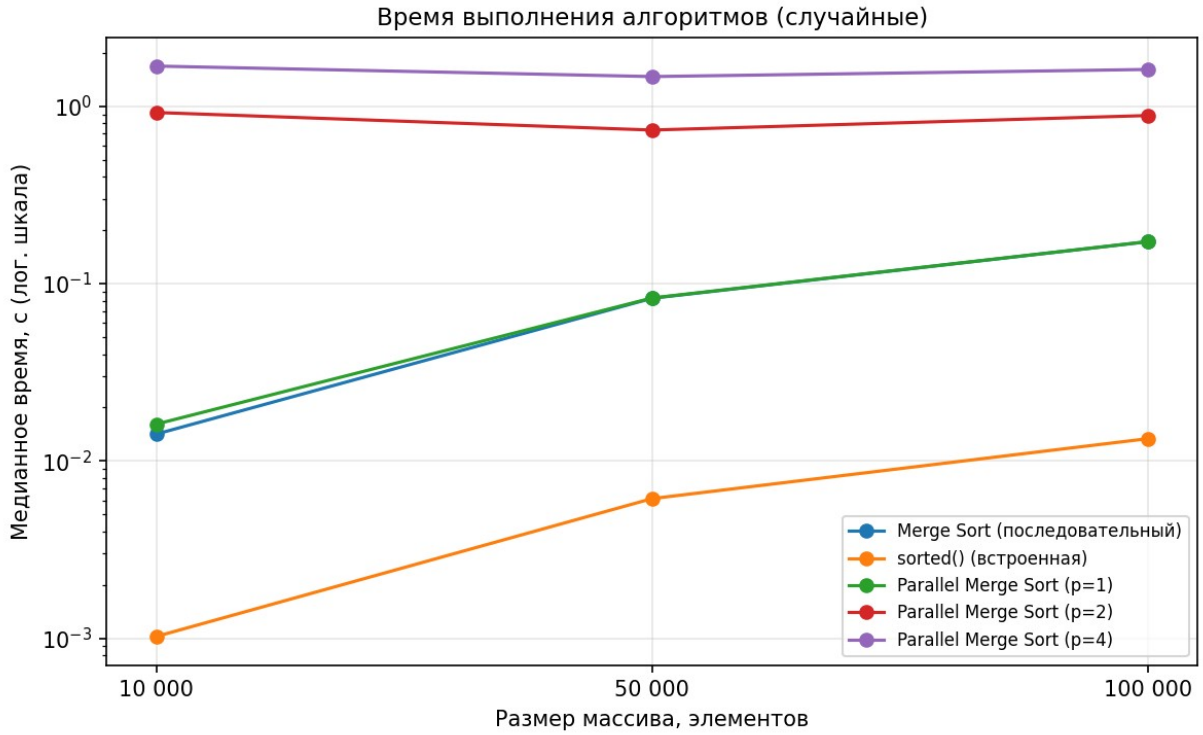
Проведено 75 измеряемых запусков: полный набор для случайных данных (3 размера × 5 конфигураций × 3 повтора = 45 запусков) и дополнительные сравнения отсортированных и обратных данных при n = 50 000 (по 15 запусков на каждый тип). Входные данные генерируются с фиксированным seed = 42. Таймер — time.perf_counter(); генерация и проверка корректности не входят в замер. Итог — медиана трёх повторов.

Параметр	Значение
Размеры массива	10 000; 50 000; 100 000 элементов
Типы данных	random на трёх размерах; sorted и reversed при n = 50 000
Алгоритмы	Merge Sort; sorted(); Parallel Merge Sort при p = 1, 2, 4
Повторы	3 на каждую конфигурацию
Метрики	S(p) = T_merge / T_parallel(p); E(p) = S(p) / p
Проверка	Все 25 строк итогового CSV имеют correct = True

8. Результаты измерений

n	Merge Sort, мс	sorted(), мс	Parallel p=2, мс	Parallel p=4, мс
10 000	14.17	1.02	921.6	1684.2
50 000	82.61	6.13	733.9	1469.6
100 000	172.23	13.35	886.1	1611.9

Единицы измерения — миллисекунды; показатели являются медианами трёх повторов.

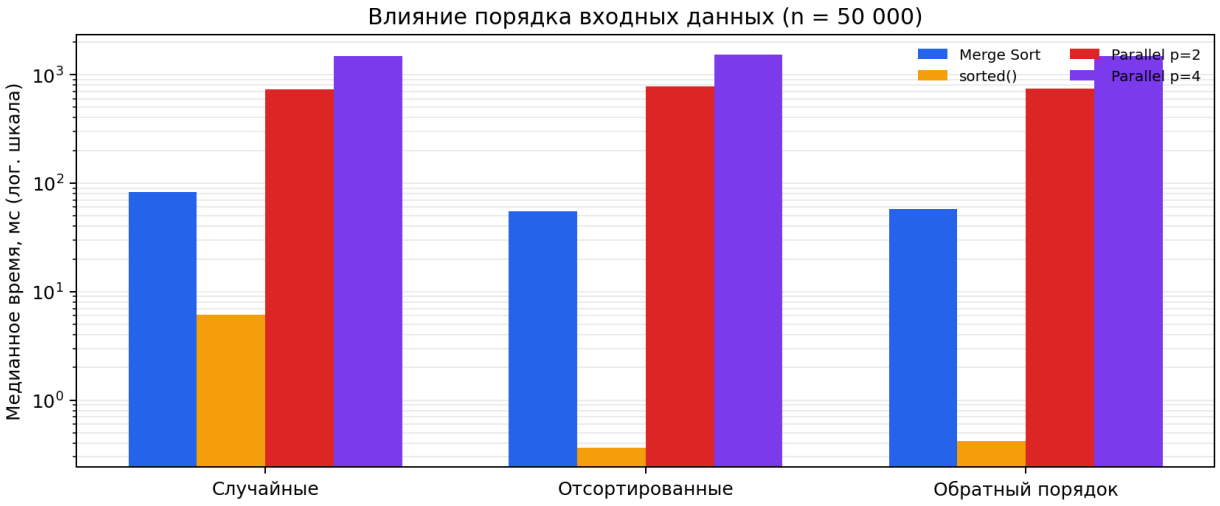


9. Влияние порядка входных данных

Для дополнительного сравнения проведены отдельные замеры на отсортированных и обратно отсортированных массивах длиной 50 000. Каждый тип сравнивался с тем же размером случайных данных, по три повтора на конфигурацию. Это сравнение не расширяет полный размерный набор: оно отвечает только на вопрос о влиянии исходного порядка при фиксированном n.

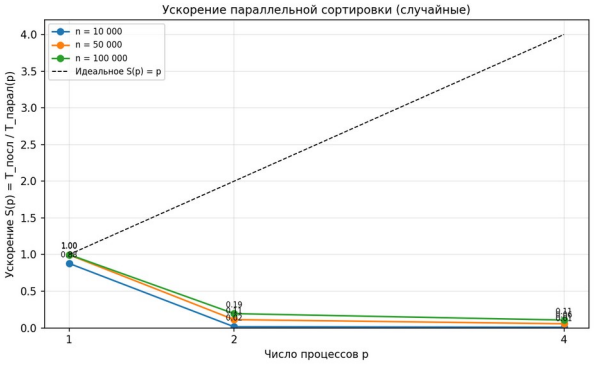
Тип данных (n = 50 000)	Merge Sort, мс	sorted(), мс	Parallel p=2, мс	Parallel p=4, мс
Случайные	82.61	6.129	733.9	1469.6
Отсортированные	55.32	0.366	782.8	1537.4
Обратный порядок	57.58	0.422	744.2	1480.4

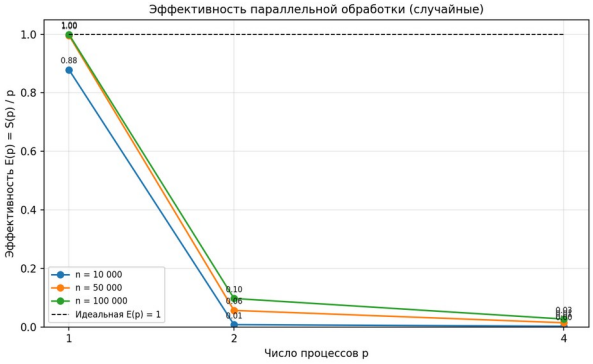
Для встроенной sorted() порядок заметно влияет на время: Timsort умеет эффективно использовать уже упорядоченные фрагменты. Для собственной Merge Sort временная сложность остаётся O(n log n), поэтому изменение порядка не должно менять класс сложности.



10. Интерпретация ускорения

В данном окружении Parallel Merge Sort с p = 2 и p = 4 медленнее последовательной версии на всех протестированных конфигурациях. Каждый замер создаёт новый пул процессов; на выбранных размерах запуск процессов, сериализация/передача списков и последовательное слияние перекрывают выигрыш от распараллеливания.





При n = 100 000 наблюдается S(2) = 0.194 и E(2) = 0.097. Так как S(p) < 1, параллельный вызов медленнее базового. Для p = 4 S(4) ≈ 0.107.

11. Выводы и ограничения

- Корректность подтверждена для всех 25 итоговых строк CSV: каждый результат совпал с sorted(вход).
- При n = 100 000 sorted() заняла 13,35 мс против 172,23 мс у Python Merge Sort; это сравнение оптимизированной встроенной реализации с учебной реализацией на Python.
- Параллельный вариант не дал ускорения на текущем стенде. Возможное улучшение — переиспользовать пул между повторами, исследовать большие массивы и сокращать стоимость передачи данных.
- Стенд: Python 3.13.5; Linux-6.18.55-x86_64-with-glibc2.41; 3 логических CPU. Это ограниченная Linux-среда, а не Mac пользователя; перед защитой рекомендуется повторить замеры локально.

Практический вывод Параллелизм имеет смысл только тогда, когда объём работы на процесс превышает накладные расходы запуска и обмена данными.

Фактические данные: results/benchmark_results.csv и results/charts/*.png. ФИО участников и реальный личный вклад заполнить перед сдачей.